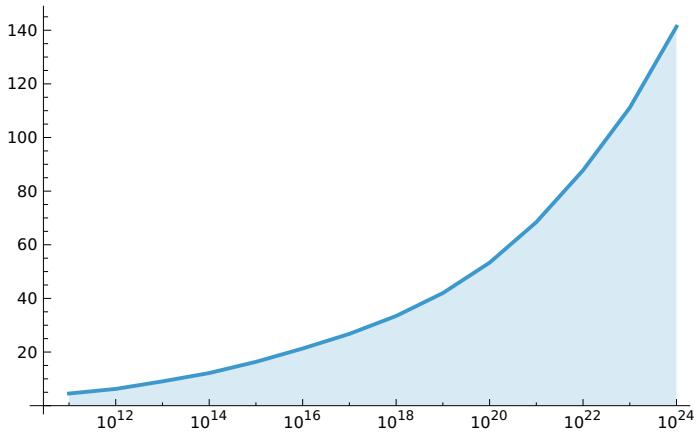
```
(* List of fast Gourdon alpha factors (alpha = alpha_y * alpha_z) found by
 running pi(x) benchmarks using the find_optimal_alpha_gourdon.sh script *)
```

In[31]:=
```
alphaGourdon = {{10 ^ 11, 4.5278}, {10 ^ 12, 6.2509},
  {10 ^ 13, 9.0645}, {10^14, 12.171}, {10^15, 16.342}, {10^16, 21.316},
  {10^17, 26.776}, {10^18, 33.448}, {10^19, 41.991}, {10^20, 53.326},
  {10^21, 68.432}, {10^22, 87.772}, {10^23, 111.122}, {10^24, 141.342}}
```

Out[31]=
```
{{100 000 000 000, 4.5278}, {1 000 000 000 000, 6.2509}, {10 000 000 000 000, 9.0645},
 {100 000 000 000 000, 12.171}, {1 000 000 000 000 000, 16.342}, {10 000 000 000 000 000, 21.316},
 {100 000 000 000 000 000, 26.776}, {1 000 000 000 000 000 000, 33.448},
 {10 000 000 000 000 000 000, 41.991}, {100 000 000 000 000 000 000, 53.326},
 {1 000 000 000 000 000 000 000, 68.432}, {10 000 000 000 000 000 000 000, 87.772},
 {100 000 000 000 000 000 000 000, 111.122}, {1 000 000 000 000 000 000 000 000, 141.342}}
```

In[32]:=
```
ListLogLinearPlot[alphaGourdon, Filling → Bottom, Joined → True]
```

Out[32]=




```
(* alpha is a tuning factor that balances the compuation of the
 easy special leaves (A + C formulas) and the hard special leaves
 (D formula). The formula below is used in the file src/util.cpp
 to calculate a fast alpha factor for the computation of pi(x). *)
```

In[33]:=
```
NonlinearModelFit[alphaGourdon, a (Log[x])^3 + b (Log[x])^2 + c Log[x] + d, {a, b, c, d}, x]
```

Out[33]=
```
FittedModel[ -237. + 21.5 «1» - «1» + 0.00695 Log[x]^3 ]
```